

Docker Essentials

Containerização Moderna para Desenvolvedores

Mini-Curso | Hands-on

Instrutor

Especialista em Arquitetura de Software

Formação & Certificações

- Visual Basic 6.0 - **SENAC**
- Bacharel em Sistemas de Informação - **ULBRA**
- MBA em Arquitetura de Software - **Instituto IGTI**
- Certificado Microsoft em Programação **C#**

Experiência Profissional

- **17 anos** de desenvolvimento de software

Objetivos de Aprendizagem

Ao final, você dominará:

Competência	O que você aprenderá
 Conceitual	Docker vs VMs, arquitetura, ecossistema
 Prático	Comandos essenciais, ciclo de vida
 Criativo	Dockerfiles, imagens customizadas
 Produtivo	Docker Compose, deploy real

Resultado: Capacidade de containerizar e deployar aplicações completas

Parte 1: Fundamentos Docker

Por que Docker existe?

"Funciona na minha máquina!" 🤔



"Funciona em qualquer lugar!" 🎯



O Problema Clássico

Cenário típico:

- Desenvolvedor: *"Aqui funciona perfeitamente!"*
- DevOps: *"Não roda em produção..."* 
- Cliente: *"Quando fica pronto?"* 

Causas:

- Diferentes versões de runtime
- Dependências conflitantes
- Configurações de ambiente distintas

✓ A Solução Docker

Cenário com Docker:

- Desenvolvedor: "*Container pronto!*"
- DevOps: "*Deploy realizado!*" ✓
- Cliente: "*Funcionando!*" 🎉

Como:

- Aplicação + dependências = container portátil
- Mesmo ambiente: dev → teste → produção
- Deploy consistente e rápido

Docker em 30 Segundos

Docker = Empacotador universal de aplicações

 **Aplicação + Dependências = Container Portátil**

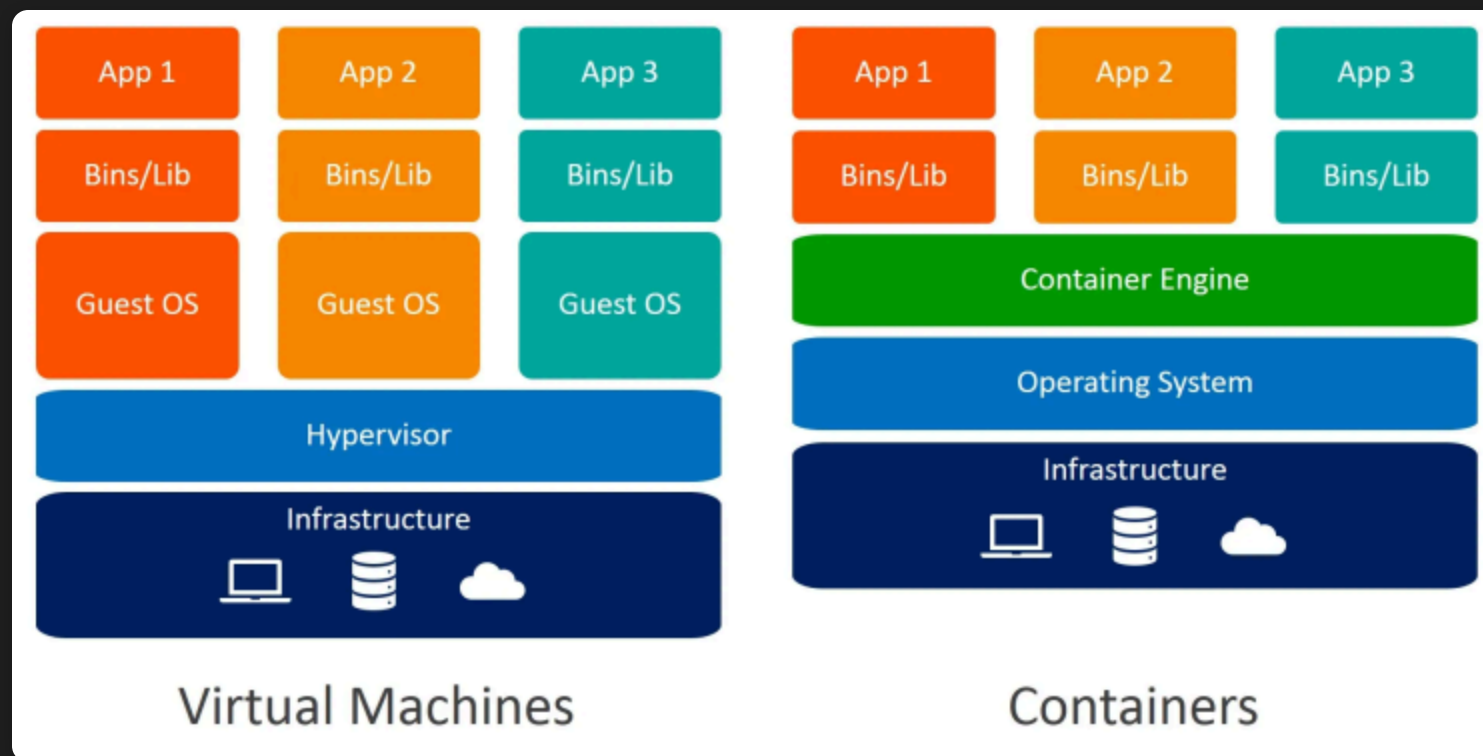
Vantagens Imediatas

-  **Velocidade**: Segundos vs minutos (VMs)
-  **Eficiência**: MBs vs GBs de overhead
-  **Consistência**: Dev = Teste = Produção
-  **Portabilidade**: Local → Cloud sem mudanças



Containers vs Virtual Machines

Aspecto	 Containers	 VMs
Startup	< 5 segundos	1-3 minutos
Memória	10-100 MB	1-4 GB
Performance	~Nativa	Overhead 10-20%
Isolamento	Processo	Sistema completo
Portabilidade	Alta	Média



Container = Processo isolado compartilhando kernel

VM = Sistema operacional completo virtualizado

Conceitos Essenciais

Image (Imagem)

Template/Receita para criar containers
Exemplo: "nginx:latest" = Servidor web pronto para uso

Container

Instância executando de uma imagem
Exemplo: Seu website rodando no nginx

Dockerfile

"Receita" para construir uma imagem personalizada
Exemplo: Como instalar sua aplicação .NET

Setup e Primeiro Container

Verificar Instalação

```
docker --version  
docker run hello-world
```

Demo Live: Servidor Web em 10 Segundos

```
docker run -d -p 8080:80 --name meu-site nginx:alpine  
curl http://localhost:8080
```

Acesse: <http://localhost:8080> 

Hands-on: Primeiro "Hello World"

Execute e observe:

```
# 1. Baixar e executar
docker run hello-world

# 2. Ver o que aconteceu
docker ps -a

# 3. Modo interativo
docker run -it ubuntu:20.04 bash
# Dentro do container: ls, pwd, exit
```

Objetivo: Entender o ciclo de vida básico

Parte 2: Containers na Prática



Comandos de Execução

```
docker run [IMAGE]           # Criar e executar
docker run -d [IMAGE]         # Background (daemon)
docker run -it [IMAGE] bash   # Interativo
docker run -p 8080:80 [IMAGE] # Mapear porta
```



Comandos de Gerenciamento

```
docker ps           # Containers ativos
docker ps -a         # Todos os containers
docker images        # Imagens disponíveis
docker stats         # Estatísticas dos containers
```

```
docker stop [CONTAINER]  # Parar
docker start [CONTAINER]  # Iniciar
docker rm [CONTAINER]     # Remover
docker logs [CONTAINER]   # Ver logs
```




Demo: Servidor Web Completo

Do Básico ao Personalizado

Passo 1: Servidor Simples

```
docker run -p 8080:80 --name meu-site nginx:alpine
```

Passo 2: Com Conteúdo Próprio

```
# Criar conteúdo
mkdir -p meu-site
echo '<h1>Meu Primeiro Docker!</h1><p>Site funcionando!</p>' > meu-site/index.html

# Servir conteúdo personalizado
docker run -d -p 8080:80 --name meu-site \
  -v $(pwd)/meu-site:/usr/share/nginx/html nginx:alpine
```

Teste: <http://localhost:8080> 

Comandos úteis:

```
docker logs meu-site    # Ver logs
docker stop meu-site    # Parar
docker rm meu-site      # Remover
```

Flags Essenciais

```
-d          # Background
-it         # Interativo
-p 8080:80  # Mapear porta
--name web  # Nome do container
-e VAR=val  # Variável ambiente
-v /data:/data # Volume mount
--rm        # Auto-remove
```

Memorize estas flags! 🧠

Persistência de Dados

✗ Problema: Dados Perdidos

```
docker run -d --name meu-banco -e POSTGRES_PASSWORD=senha123 postgres:13
docker exec -it meu-banco createdb -U postgres meu_app_senac
docker exec -it meu-banco psql -U postgres -l    # Listar bases criadas
docker rm -f meu-banco    # 💀 Dados perdidos!
```

✓ Solução: Volumes

```
# Opção 1: Volume nomeado (Docker gerencia)
docker volume create meus-dados
docker run -d \
  --name meu-banco \
  -e POSTGRES_PASSWORD=senha123 \
  -v meus-dados:/var/lib/postgresql/data \
  postgres:13 # 🎉 Dados persistem!
```

```
# Opção 2: Mapeamento direto para pasta específica
mkdir -p $(pwd)/data/postgresql
docker run -d \
  --name meu-banco \
  -e POSTGRES_PASSWORD=senha123 \
  -v $(pwd)/data/postgresql:/var/lib/postgresql/data \
  postgres:13 # 💡 Dados na pasta escolhida!
```

Lab: Persistência de Dados

Objetivo: Site que "lembra" das mudanças

```
# Criar pasta permanente
mkdir -p meu-site
echo '<h1>Meu Site Persistente</h1><p>Editavel!</p>' > meu-site/index.html

# Container com volume persistente
docker run -d --name meu-site \
  -p 8080:80 \
  -v $(pwd)/meu-site:/usr/share/nginx/html \
  nginx:alpine

# Editar arquivo (site muda instantaneamente!)
echo '<h1>Site Atualizado \o/ !</h1><p>\o/</p>' > meu-site/index.html
```

Experimente:

1.  Acesse <http://localhost:8080>
2.  Edite o arquivo `meu-site/index.html`
3.  Atualize o navegador - mudança aparece!
4.  Pare e inicie o container - dados persistem!



Parte 3: Construindo Imagens

De Consumidor a Criador

Até agora: Usamos imagens prontas (`nginx`, `postgres`)

Agora: Vamos criar nossas próprias imagens! 🚀

Conceito:

- Imagem = 📄 Receita de bolo
- Container = 🍰 Bolo pronto

Dockerfile: Explorando Possibilidades

Principais Instruções Dockerfile

Comando	Função	Exemplo
FROM	Imagem base	<code>FROM node:18-alpine</code>
COPY	Copiar arquivos	<code>COPY . /app</code>
RUN	Executar comandos	<code>RUN npm install</code>
WORKDIR	Diretório de trabalho	<code>WORKDIR /app</code>
EXPOSE	Expor porta	<code>EXPOSE 3000</code>
CMD	Comando padrão	<code>CMD ["npm", "start"]</code>



Dockerfile: Primeira "Receita"

Objetivo: Hospedar uma página HTML simples

Arquivo: `index.html`

```
<!DOCTYPE html>
<html>
<head>
  <title>Minha App Docker!</title>
  <style>
    body { background: #1e1e1e; color: #fff; text-align: center; padding: 50px; }
    h1 { color: #4ec9b0; font-size: 2.5em; }
  </style>
</head>
<body>
  <h1>Primeira Pagina Docker!</h1>
  <p>Container funcionando!</p>
</body>
</html>
```

Arquivo: **Dockerfile**

```
FROM nginx:alpine
COPY index.html /usr/share/nginx/html/
EXPOSE 80
```

3 passos simples: Base (nginx) → Copiar HTML → Expor porta

Build e Teste

```
# Construir imagem
docker build -t minha-pagina:v1.0 .

# Executar container
docker run -d -p 8080:80 --name meu-site minha-pagina:v1.0

# Testar no navegador
echo "Acesse: http://localhost:8080"
```

Magic! ✨ Sua página rodando em container!

Abra o navegador: <http://localhost:8080>

Cache do Dockerfile: Otimização Inteligente

Como funciona o Cache?

Docker reutiliza camadas quando possível:

- Cada linha do Dockerfile = 1 camada
- Se a linha não mudou → reutiliza cache
- Se mudou → reconstrói desta linha em diante

❌ Ineficiente vs ✅ Otimizado

❌ Problema	✅ Solução
Copia tudo junto	Dependências primeiro
<code>COPY . /app</code>	<code>COPY package.json /app</code>
Sempre reinstala	Cache inteligente
Reinstala mesmo sem mudanças	Cache se package.json não mudou
Código primeiro	Código por último
Quebra cache desnecessariamente	Preserva cache das dependências

Resultado: Build 10x mais rápido! ⚡

💡 Exemplo Prático: Node.js App

❌ Ineficiente (sempre reinstala dependências):

```
FROM node:18-alpine
COPY . /app          # Copia TUDO (código + package.json)
WORKDIR /app
RUN npm install       # Roda sempre, mesmo se só código mudou
CMD ["npm", "start"]
```

✅ Otimizado (cache inteligente):

```
FROM node:18-alpine
WORKDIR /app
COPY package.json .   # Só o arquivo de dependências
RUN npm install        # Cache preservado se package.json não mudou
COPY src/ ./src/       # Código por último (muda mais frequentemente)
CMD ["npm", "start"]
```

Diferença: Segunda build em 5 segundos ao invés de 2 minutos! 🚀



Docker Registry: Onde Ficam as Imagens

Tipos de Registry

Tipo	Descrição	Exemplos	Uso
Público	Acesso aberto para todos	Docker Hub, Quay.io	Imagens open-source
Privado	Acesso restrito/autenticado	AWS ECR, Azure ACR	Aplicações corporativas
Local	Servidor próprio da empresa	Harbor, Nexus	Controle total, segurança



Docker Hub: Registry Público Principal

O que é:

- Maior registro público de imagens Docker
- Milhões de imagens prontas para uso
- Gratuito para repositórios públicos

Exemplos práticos:

```
# Imagens oficiais (sem prefixo)
docker pull nginx:alpine
docker pull postgres:13
docker pull node:18

# Imagens da comunidade (com usuário/)
docker pull bitnami/wordpress
docker pull grafana/grafana
```

Registry Privado: Segurança Corporativa

Por que usar Registry Privado?

-  **Segurança:** Código privado
-  **Controle:** Permissões de acesso
-  **Performance:** Infraestrutura local/cloud
-  **Compliance:** Políticas da empresa

Principais Providers

 **Cloud:** AWS ECR • Azure ACR • Google GCR

 **Local:** Harbor • GitLab Registry • Nexus

Comandos Essenciais para Registry

Docker Hub (Público)

```
# Login no Docker Hub
docker login

# Tag da imagem para seu usuário
docker tag minha-app meuusuario/minha-app:v1.0

# Push para Docker Hub
docker push meuusuario/minha-app:v1.0

# Pull de qualquer lugar
docker pull meuusuario/minha-app:v1.0
```

Registry Privado (exemplo AWS ECR)

```
# Login no registry privado
aws ecr get-login-password | docker login --username AWS --password-stdin 123456789.dkr.ecr.us-east-1.amazonaws.com

# Tag para registry privado
docker tag minha-app 123456789.dkr.ecr.us-east-1.amazonaws.com/minha-app:v1.0

# Push para registry privado
docker push 123456789.dkr.ecr.us-east-1.amazonaws.com/minha-app:v1.0
```

💡 **Dica:** Registry privado = controle total sobre suas aplicações! 🎯

Parte 4: Orquestração com Compose

Problema dos Múltiplos Containers

```
# App completa = múltiplos containers
docker run -d --name postgres ...
docker run -d --name redis ...
docker run -d --name api --link postgres ...
docker run -d --name frontend --link api ...
```

Problemas:

- 🤔 Muitos comandos
- 🔗 Dependências complexas
- 🌐 Configuração manual de rede

✨ Docker Compose: Possibilidades

O que você pode fazer com Compose? 🚀

Recurso	Descrição	Exemplo
Multi-containers	Vários serviços juntos	Web + DB + Cache
Networks	Comunicação entre containers	<code>app-network</code>
Volumes	Dados persistentes	<code>db-data:/var/lib/mysql</code>
Environment	Variáveis por serviço	<code>NODE_ENV=production</code>
Dependencies	Ordem de inicialização	<code>depends_on: [database]</code>
Scaling	Múltiplas instâncias	<code>docker compose up --scale api=3</code>

Principais Vantagens

-  **Declarativo**: Descreve o que quer
-  **Reproduzível**: Mesmo resultado sempre
-  **Rápido**: Um comando sobe tudo
-  **Limpo**: Um comando remove tudo

1 arquivo YAML = Stack completa! 

docker-compose.yml: Primeira Stack

```
services:
  database:
    image: postgres:13
    environment:
      POSTGRES_DB: meuapp
      POSTGRES_PASSWORD: dev123
    ports: ["5432:5432"]
    volumes: ["db_data:/var/lib/postgresql/data"]
    restart: unless-stopped

  api:
    build: ./api
    depends_on: [database]
    ports: ["3000:3000"]
    environment:
      DATABASE_URL: postgresql://dev123@database/meuapp
    restart: unless-stopped

networks:
  default:
    name: app-network

volumes:
  db_data:
```

💡 Recursos adicionados:

- `depends_on`: Portfolio depende de Blog e Homepage, Blog depende de Homepage
- `restart: unless-stopped`: Containers reiniciam automaticamente se falharem
- `networks`: Rede customizada para comunicação entre containers

🔍 Sobre Restart Automático:

! **IMPORTANTE:** `restart: unless-stopped` funciona apenas para **falhas não intencionais**:

-  **Reinicia:** Crash, erro da aplicação, falha de sistema
-  **NÃO reinicia:** `docker stop` manual (parada intencional)

Políticas de Restart:

- `no`: Nunca reinicia (padrão)
- `always`: Sempre reinicia (mesmo após docker stop)
- `unless-stopped`: Reinicia até você parar manualmente
- `on-failure`: Só reinicia em caso de erro

Comandos Essenciais do Compose

```
# Subir toda a stack
docker compose up -d

# Ver logs de todos os serviços
docker compose logs -f

# Parar tudo
docker compose down

# Parar e remover volumes
docker compose down -v

# Reiniciar
docker compose restart
```

Magic Command: `docker compose up -d` ✨

Evolução Conquistada

Antes	Depois
😵 "Docker é complexo"	🎯 "Docker é essencial"
👤 Dependente de DevOps	🚀 Deploy independente
💻 "Só funciona aqui"	🌍 "Funciona em qualquer lugar"
🕒 Configuração demorada	⚡ Setup em segundos

Você evoluiu! 🚀

⚡ Cheat Sheet - Comandos do Dia a Dia

🏃 EXECUTAR CONTAINERS

```
docker run -d -p 3000:80 --name meu-app nginx:alpine
docker run -it --rm ubuntu bash           # Interativo e temporário
docker run -v $(pwd):/app nginx:alpine   # Com volume
```

👁 MONITORAR

```
docker ps           # Containers rodando
docker logs -f meu-app # Logs em tempo real
docker exec -it meu-app sh # Entrar no container
```

🛠 GERENCIAR

```
docker stop meu-app && docker rm meu-app # Parar e remover
docker system prune -f                   # Limpeza geral
```

🏗 CONSTRUIR IMAGENS

```
docker build -t minha-app:latest .           # Build básico
docker images | grep minha-app                # Ver suas imagens
```

📋 DOCKER COMPOSE

```
docker compose up -d           # Subir stack
docker compose down -v          # Derrubar tudo + volumes
```

Salve este cheat sheet! 📋

Próximos Passos (Roadmap)

Intermediário

- Docker Swarm & Kubernetes
- Registry privado
- Multi-stage optimization

Produção & Segurança

- Secrets management
- Security scanning & Monitoring
- CI/CD (GitHub Actions)

Cloud & Scale

- AWS ECS/EKS
- Azure Container Instances
- Google Cloud Run

Continue Aprendendo, Continue Crescendo!

"Docker não é apenas uma ferramenta, é uma revolução no desenvolvimento moderno."

 Contato



Dúvidas? Vamos conversar! 